

Numerical Addendum: LarsenHBond Calculator

Generated/Reviewed Documentation (Codex/Opus/Human) for v0.0.1

The LarsenHBond calculator is an empirical tensor-grid lookup followed by a frame change and a spherical-tensor decomposition. The lookup key is a donor class, an acceptor class, and the three geometry coordinates (r, θ, ρ) . LarsenHBond's output is already in ppm: the grid entries are DFT shielding-difference tensors from the Larsen parameterization, not a geometric kernel awaiting calibration. The listings below are distilled from the source, with loops and bookkeeping trimmed, but the constants, guards, inequalities, and multiplication order are the implementation's.

1 The lookup point

The donor sweep first makes a finite candidate set. Amide hydrogens and alpha hydrogens are donor candidates; acceptor oxygen candidates come from the atom spatial index within 4.2 Å of the donor hydrogen. The acceptor classifier supplies the acceptor class and the two acceptor-side atoms used below. For a classified candidate, the code computes (r, θ, ρ) , rejects $\theta < 90^\circ$ and $\theta > 180^\circ$, then asks the grid for an interpolated record. Equality at 90° and 180° reaches the grid lookup.

```
constexpr double kSpatialCutoff_A = 4.2;
constexpr double kThetaMinDeg     = 90.0;
constexpr double kThetaMaxDeg     = 180.0;

auto candidate_atoms =
    spatial.AtomsWithinRadius(donor_pos, kSpatialCutoff_A);

LarsenHBondGeometry geom = ComputeLarsenHBondGeometry(
    donor_H_pos, accept_O_pos, accept_C_pos, accept_thd_pos);

if (geom.theta_deg < kThetaMinDeg || geom.theta_deg > kThetaMaxDeg)
    return PairResult::ThetaOutOfRange;

LarsenHBondRecord rec = grid.QueryNearest(donor_class, acc.class_, geom);
if (!rec.IsHit()) return PairResult::GridMiss;
```

The spatial cutoff, the two theta limits, and the water offset $\Delta\sigma_w = 2.07$ ppm (the isotropic term added later to amide hydrogens for which the sweep finds no θ -accepted geometric pair) are compiled constants in this calculator. The H-bond keys in `data/calculator_params.toml` belong to the separate kernel-form hydrogen-bond calculators and are not read by this grid lookup.

In the geometry helper, h is the donor hydrogen, o the acceptor oxygen, c the heavy atom bonded to the acceptor oxygen, and t the acceptor-side third atom. The distance is $r = \|o - h\|$. The angle θ is the H–O–C angle at the acceptor oxygen. The cosine is clamped to $[-1, 1]$ before `arccos`, so a rounding excursion at the dot-product boundary becomes the endpoint angle rather than a NaN.

The dihedral ρ compares the H–O–C plane to the O–C–third plane. The two cross products make the plane normals, and the `atan2` pair keeps the sign and quadrant. The value is in degrees and is not negated before lookup; the dense grids are keyed on this same sign convention.

```
Vec3 H_to_O = acceptor_O_pos - donor_H_pos;
geom.r_angstrom = H_to_O.norm();

Vec3 O_to_H = donor_H_pos - acceptor_O_pos;
Vec3 O_to_C = acceptor_C_pos - acceptor_O_pos;
double cos_theta = O_to_H.dot(O_to_C) /
    (O_to_H.norm() * O_to_C.norm());
cos_theta = std::clamp(cos_theta, -1.0, 1.0);
```

```

geom.theta_deg = std::acos(cos_theta) * (180.0 / M_PI);

Vec3 C_to_third = acceptor_third_pos - acceptor_C_pos;
Vec3 n1 = H_to_O.cross(O_to_C);           // plane H-O-C
Vec3 n2 = O_to_C.cross(C_to_third);       // plane O-C-third
Vec3 m1 = n1.cross(O_to_C.normalized());
double x = n1.dot(n2);
double y = m1.dot(n2);
geom.rho_deg = std::atan2(y, x) * (180.0 / M_PI);

```

The donor and acceptor classes select one of six archives. Side-chain carbonyl acceptors fold into the backbone-carbonyl archive because the Larsen grid set has no separate side-chain-amide acceptor scan. In accumulation, the acceptor-side secondary readouts are routed only for backbone-carbonyl acceptors with a valid downstream residue. The archive choice selects the tensor arrays; the (r, θ, ρ) coordinates select the cell inside that archive.

```

bool nma_acceptor =
    acceptor == HBondAcceptorClass::BackboneCarbonyl ||
    acceptor == HBondAcceptorClass::SidechainCarbonyl;

if (donor == HBondDonorClass::AmideHydrogen) {
    if (nma_acceptor) return kArchiveNMANMA;
    if (acceptor == HBondAcceptorClass::HydroxylOxygen) return kArchiveNMACOH;
    if (acceptor == HBondAcceptorClass::CarboxylateOxygen) return kArchiveNMACOO;
} else {
    if (nma_acceptor) return kArchiveALANMA;
    if (acceptor == HBondAcceptorClass::HydroxylOxygen) return kArchiveALACOH;
    if (acceptor == HBondAcceptorClass::CarboxylateOxygen) return kArchiveALACOO;
}

```

2 Trilinear tensor interpolation

Each readout dataset is a flat row-major array with logical shape $(N_r, N_\theta, N_\rho, 3, 3)$. A successful query first locates a lower index and a fractional coordinate on each axis. The r and θ axes are bounded axes. The tolerance is 10^{-9} of one axis step: values below the lower bound by more than that, or above the upper bound by more than that, return a non-hit. Values inside that tolerance are clamped onto the stored bound, so the lower bound has fraction 0 and the upper bound has fraction 1 in the last cell.

The ρ axis is different. It is periodic, with a required 360° period. The record first wraps ρ into $[-180^\circ, 180^\circ)$; the periodic axis lookup then locates the cell in the half-open period. In the current dense grids the stored ρ axis runs from -180° to 175° in 5° steps, so the upper corner of the last cell wraps to index 0. A value near 180° blends the 175° plane with the -180° plane; 180° itself wraps to -180° .

```

rec.rho_deg = WrapRho(geom.rho_deg);

AxisLookup lr =
    LookupAxis(g.r_axis,      geom.r_angstrom, false);
AxisLookup lth =
    LookupAxis(g.theta_axis,  geom.theta_deg,   false);
AxisLookup lrho =
    LookupAxis(g.rho_axis,    rec.rho_deg,      true);

if (lr.idx < 0 || lth.idx < 0 || lrho.idx < 0)
    return rec;                               // non-hit

```

```

// Non-periodic path, distilled from LookupAxis.
double step = axis[1] - axis[0];

```

```

double tol = std::abs(step) * kAxisBoundTolerance; // 1e-9
if (value < axis.front() - tol || value > axis.back() + tol)
    return miss;

double v = std::clamp(value, axis.front(), axis.back());
double f = (v - axis.front()) / step;
int i = static_cast<int>(std::floor(f));
i = std::min(i, n - 2);
i = std::max(i, 0);

out.idx      = i;
out.idx_next = i + 1;
out.frac     = std::clamp(f - i, 0.0, 1.0);

```

```

// Periodic path, distilled from LookupAxis.
double step = axis[1] - axis[0];
double period = axis.back() - axis[0] + step;
if (std::abs(period - 360.0) > 1e-6) return miss;

double w = axis[0] + std::fmod(value - axis[0], period);
if (w < axis[0]) w += period;

double f = (w - axis[0]) / step;
int i = static_cast<int>(std::floor(f));
int i_next = i + 1;
if (i_next >= n) i_next = 0; // last rho cell wraps

out.idx      = i;
out.idx_next = i_next;
out.frac     = f - i;

```

The interpolation itself is done independently for each matrix entry. For a fixed row and column, the code reads the eight corner values $c_{\alpha\beta\gamma}$, blends first along ρ , then along θ , then along r . This is the same product-weight blend as

$$\sigma_{ab}(r, \theta, \rho) = \sum_{\alpha, \beta, \gamma \in \{0,1\}} (1 - f_r)^{1-\alpha} f_r^\alpha (1 - f_\theta)^{1-\beta} f_\theta^\beta (1 - f_\rho)^{1-\gamma} f_\rho^\gamma G_{i+\alpha, j+\beta, k+\gamma, a, b}.$$

The eight weights sum to one for each matrix entry. The dense HDF5 grids contain the cubic-spline pre-computation; the runtime path is trilinear.

```

// Distilled from TrilinearMat3, one row/col entry at a time.
double c000 = cell(ir,      ith,      irho);
double c001 = cell(ir,      ith,      irho_next);
double c010 = cell(ir,      ith_next, irho);
double c011 = cell(ir,      ith_next, irho_next);
double c100 = cell(ir_next, ith,      irho);
double c101 = cell(ir_next, ith,      irho_next);
double c110 = cell(ir_next, ith_next, irho);
double c111 = cell(ir_next, ith_next, irho_next);

double c00 = c000 * (1.0 - frho) + c001 * frho;
double c01 = c010 * (1.0 - frho) + c011 * frho;
double c10 = c100 * (1.0 - frho) + c101 * frho;
double c11 = c110 * (1.0 - frho) + c111 * frho;

double c0 = c00 * (1.0 - fth) + c01 * fth;
double c1 = c10 * (1.0 - fth) + c11 * fth;

out(row, col) = c0 * (1.0 - fr) + c1 * fr;

```

When a validity mask is present, the same eight corners are inspected. If any corner has mask value zero, `rec.any_corner_imputed` is set. The flag does not change the interpolation weights; it records that the returned tensor used at least one dense cell filled upstream for continuity.

3 The donor frame

The grid tensors are stored in a donor-centered canonical frame. The runtime rebuilds that frame from the protein-side donor atoms before rotating any looked-up tensor. For an amide donor, the anchor is the amide N and the third atom is the predecessor residue’s carbonyl C. For an alpha donor, the anchor is C_α and the third atom is the same residue’s N.

The rotation matrix R stores the canonical basis vectors as rows in the protein lab coordinates. The z -axis is fixed first: $\hat{z} = (\mathbf{h} - \mathbf{a}) / \|\mathbf{h} - \mathbf{a}\|$. Then $\mathbf{q} = \mathbf{h} - \mathbf{u}$, with $\mathbf{u}$ the donor-side third atom, loses its $\hat{z}$ component:

$$\mathbf{x}_{\text{raw}} = \mathbf{q} - (\mathbf{q} \cdot \hat{z})\hat{z}.$$

Normalizing that projected vector gives $\hat{x}$, and $\hat{y} = \hat{z} \times \hat{x}$. The three guards test norms with a strict $< 10^{-9}$ comparison. If a guard fires, the helper returns the identity matrix and logs a warning; the caller then uses that identity in the same rotation path as any other R .

```
constexpr double kTinyVec = 1e-9;

Vec3 z_raw = donor_H_pos - donor_anchor_pos;
if (z_raw.norm() < kTinyVec) return Mat3::Identity();
Vec3 z = z_raw.normalized();

Vec3 third_to_H = donor_H_pos - donor_third_pos;
if (third_to_H.norm() < kTinyVec) return Mat3::Identity();

Vec3 x_raw = third_to_H - (third_to_H.dot(z)) * z;
if (x_raw.norm() < kTinyVec) return Mat3::Identity();

Vec3 x = x_raw.normalized();
Vec3 y = z.cross(x);

Mat3 R;
R.row(0) = x;
R.row(1) = y;
R.row(2) = z;
```

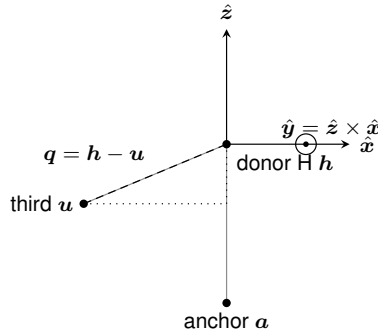


Figure 1: The donor frame keeps the anchor-to-H direction as $\hat{z}$. The vector $\mathbf{h} - \mathbf{u}$ supplies the second direction; one projection removes its $\hat{z}$ component before normalization gives $\hat{x}$. The $\hat{y}$ axis is the cross product $\hat{z} \times \hat{x}$.

4 Rotating the tensor

The stored-frame convention fixes the multiplication direction. During grid generation, each log tensor was stored as

$$\sigma_{\text{canon}} = R_{\log} \sigma_{\log} R_{\log}^T.$$

The protein-side R_p has the same row convention, so recovering the protein lab-frame tensor puts the transpose on the left:

$$\sigma_{\text{lab}} = R_p^T \sigma_{\text{canon}} R_p.$$

Both indices of the 3×3 shielding tensor move through the same frame change. The code applies this helper to donor-side and acceptor-side readouts; the frame is still the donor frame for both, because the archive stores every readout in that canonical donor basis.

```
Mat3 RotateTensorToProteinLabFrame(
    const Mat3& sigma_canonical,
    const Mat3& R_protein) {
    return R_protein.transpose() * sigma_canonical * R_protein;
}

Mat3 R_protein = ComputeLarsenDonorFrame(
    donor_H_pos, donor_anchor, donor_third);

Mat3 sigma_lab = RotateTensorToProteinLabFrame(
    readout.canonical_tensor, R_protein);

AccumulateContribution(conf.MutableAtomAt(target_ai),
    term, sigma_lab);
```

The four pair-term accumulators are 3×3 sums in ppm. At the end of the conformation, the total pair tensor for an atom is the sum of the primary and secondary amide-H terms and the primary and secondary alpha-H terms. The water term is a separate scalar field on amide H atoms; it is not added to the tensor total.

```
Mat3 shielding = a.larsen_hbond_1pHB_tensor
    + a.larsen_hbond_2pHB_tensor
    + a.larsen_hbond_1pHaB_tensor
    + a.larsen_hbond_2pHaB_tensor;

a.larsen_hbond_shielding_tensor = shielding;
a.larsen_hbond_shielding_spherical =
    SphericalTensor::Decompose(shielding);
```

5 The shared tensor split

Every nine-column Larsen shielding array written by `WriteFeatures()` is produced by the shared decomposition below. That function is closed-form arithmetic on the matrix as accumulated; it does not pre-symmetrize or diagonalize the matrix. The trace average gives T_0 , the antisymmetric part gives T_1 , and the symmetric traceless part gives the five T_2 numbers in the shared packed order.

```
st.T0 = sigma.trace() / 3.0;

st.T1[0] = 0.5 * (sigma(1,2) - sigma(2,1));
st.T1[1] = 0.5 * (sigma(2,0) - sigma(0,2));
st.T1[2] = 0.5 * (sigma(0,1) - sigma(1,0));

double Sxx = sigma(0,0) - st.T0;
double Syy = sigma(1,1) - st.T0;
double Szz = sigma(2,2) - st.T0;
double Sxy = 0.5 * (sigma(0,1) + sigma(1,0));
double Sxz = 0.5 * (sigma(0,2) + sigma(2,0));
double Syz = 0.5 * (sigma(1,2) + sigma(2,1));

st.T2[0] = std::sqrt(2.0) * Sxy;
st.T2[1] = std::sqrt(2.0) * Syz;
```

```

st.T2[2] = std::sqrt(3.0 / 2.0) * Szz;
st.T2[3] = std::sqrt(2.0) * Szz;
st.T2[4] = (Sxx - Syy) / std::sqrt(2.0);

```

The T_2 normalization is isometric. An off-diagonal entry such as S_{xy} appears twice in the symmetric matrix, so it contributes $2S_{xy}^2$ to the Frobenius norm. The stored value $\sqrt{2} S_{xy}$ squares to that same amount. The two diagonal entries $\sqrt{3/2} S_{zz}$ and $(S_{xx} - S_{yy})/\sqrt{2}$ carry the two diagonal degrees of freedom left after $S_{xx} + S_{yy} + S_{zz} = 0$. Thus a dot product of two stored T_2 five-vectors is the Frobenius inner product of the symmetric traceless matrices they encode.

For LarsenHBond, the grid can return full 3×3 shielding-difference tensors, so all three channels can be populated. `WriteFeatures()` emits six `SphericalTensor`-packed float64 arrays of shape $N \times 9$: the total pair shielding, the four pair-term shieldings, and the $C\beta$ diagnostic shielding. It also emits the scalar water term as a length- N float64 vector in ppm and the per-atom pair count as a length- N int32 vector.

6 Glossary

Trilinear interpolation. A point inside a grid cell borrows from its eight surrounding corners, the nearer corners weighing more (the eight corner values blended by the point's three fractional distances). For one Larsen tensor entry, the code blends the two ρ corners, blends those results along θ , then blends the two remaining values along r . Formally, it is the tensor-product linear interpolant on a rectangular cell, with weights $(1 - f_r)^{1-\alpha} f_r^\alpha (1 - f_\theta)^{1-\beta} f_\theta^\beta (1 - f_\rho)^{1-\gamma} f_\rho^\gamma$.

Periodic lookup axis. A ruler bent into a ring, so the cell past its last mark uses the first mark as its upper corner. In the Larsen ρ grid, a value near 180° uses the cell from 175° to -180° , while r and θ outside their tolerated bounds return a non-hit. Formally, the coordinate is reduced modulo the period before the lower index and fractional coordinate are computed.

Gram–Schmidt donor frame. Three atoms raise a set of mutually perpendicular axes — one bond fixes the first, a second bond is straightened perpendicular to it to set the next, and their cross product gives the third (an orthonormal frame: the z -axis, then the projected x -axis, then $\hat{y} = \hat{z} \times \hat{x}$). Here $\mathbf{h} - \mathbf{a}$ fixes $\hat{z}$, and the component of $\mathbf{h} - \mathbf{u}$ perpendicular to $\hat{z}$ fixes $\hat{x}$. Formally, this is one Gram–Schmidt projection followed by $\hat{y} = \hat{z} \times \hat{x}$, with identity returned on the three guarded degeneracies.

Signed dihedral. A signed dihedral measures how one plane turns into the next around their shared bond. In the H–O–C–third coordinate, the normal to H–O–C and the normal to O–C–third are compared with `atan2`, so crossing the reference plane changes the sign of ρ . Formally, with $\mathbf{n}_1 = (\mathbf{o} - \mathbf{h}) \times (\mathbf{c} - \mathbf{o})$ and $\mathbf{n}_2 = (\mathbf{c} - \mathbf{o}) \times (\mathbf{t} - \mathbf{c})$, $\rho = \text{atan2}((\mathbf{n}_1 \times \widehat{\mathbf{c} - \mathbf{o}}) \cdot \mathbf{n}_2, \mathbf{n}_1 \cdot \mathbf{n}_2)$.

Spherical tensor split. A 3×3 tensor holds three kinds of content that rotate without mixing — an orientation-free size (the scalar trace, T_0), an axial twist from its antisymmetric part (the three T_1 components), and a five-number traceless shape for the rest of the direction-dependence (the five T_2 components). A Larsen grid tensor can carry an isotropic shielding difference in T_0 , an antisymmetric residue in T_1 , and anisotropic shielding in T_2 . Formally, it is the closed-form split of a 3×3 tensor's nine components into dimensions $1 + 3 + 5$, with the T_2 basis normalized to preserve the Frobenius norm.